Computer Sciences I
Semester 2026-I
Course Project Description

Eng. Carlos Andrés Sierra, M.Sc.
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Overview

This course project integrates the two foundational pillars of Computer Sciences I:

i. **First Half (Weeks 1–8): Algorithm Foundations** — Design, analysis, and implementation of classical algorithms including greedy strategies, backtracking, searching, sorting, and introductory complexity analysis.

ii. **Second Half (Weeks 9–16): Data Structures** — Abstract data types, linear structures (lists, stacks, queues), tree structures (BST, AVL, B-Trees), and advanced complexity analysis using asymptotic notation.

Each team will build an **interactive grid-based videogame** that serves as a living laboratory for the concepts studied throughout the semester. The game is not a metaphor for the algorithms — the algorithms *are* the game mechanics. Every new concept learned in class unlocks a new feature in the project.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavirguezs@udistrital.edu.co*.

Project Architecture

All teams share the same technical architecture, divided by programming language responsibility:

Layer	Language	Responsibility
Core Engine	C++	Linked list and BST/AVL tree implementations. Manages game-state data structures, reads `input.json`, writes `state.json`.
Algorithm & UI	Python	Greedy and backtracking algorithm logic. Pygame grid rendering, HUD, and user interaction.
Integration Bridge	JSON Files	`input.json` (Python → C++) carries player actions. `state.json` (C++ → Python) carries updated game state for rendering.

Project Folder Structure

All teams must organize their repository following this structure:

```
project/
    engine/                 C++
        linked_list.cpp
        tree.cpp            (BST or AVL, depending on variant)
        main.cpp            (reads input.json -> writes state.json)
    game/                   Python
        algorithms/
            greedy.py
            backtracking.py
        ui/
            main.py         (Pygame grid + HUD)
        bridge.py           (file I/O: reads state.json, writes input.json)
    data/
        input.json
        state.json
```

The 16 Game Variants

Each team is assigned one unique game variant. All variants share the same technical skeleton and grading rubric — only the domain and theme differ. The table below summarizes the assignment of algorithms and data structures per variant.

#	Game	Greedy (Python)	Backtracking (Python)	C++ List	C++ Tree
1	Tower Siege	Best tower placement by threat level	Enemy path avoiding active towers	Enemy wave queue	Tower registry (BST)
2	Plague Containment	Vaccinate highest-risk neighbors first	Quarantine zone without splitting city	Infection chain list	District risk tree (AVL)
3	Asteroid Miner	Nearest/highest-value asteroid first	Optimal route within fuel budget	Route history list	Asteroid catalog (BST)
4	City Builder	Place highest-happiness building first	Valid layout satisfying zoning rules	Building adjacency list	Happiness score tree (BST)
5	Ecosystem Survival	Feed most endangered species first	Migration route avoiding predators	Migration path list	Population tree (AVL)
6	Fire Brigade	Deploy unit to largest active fire first	Evacuation route avoiding fire spread	Unit dispatch queue	Zone risk tree (BST)
7	Treasure Hunt	Grab nearest highest-value item first	Find all items within step limit	Movement history list	Treasure value tree (BST)
8	Traffic Control	Green-light highest-traffic road first	Route avoiding all red-light constraints	Vehicle queue per road	Intersection load tree (AVL)
9	Hospital Triage	Treat most critical patient first	Assign patients to rooms with constraints	Patient waiting list	Severity tree (AVL)
10	Power Grid	Connect cheapest available node first	Valid network with no overloaded node	Cable connection list	Node capacity tree (BST)
11	Rescue Mission	Reach most survivors first	Path avoiding collapsed zones	Rescue route list	Survivor priority tree (AVL)
12	Market Trader	Buy cheapest available resource first	Portfolio selection within budget	Transaction history list	Price index tree (BST)

13	Ant Colony	Follow strongest pheromone trail	Find path home avoiding dead ends	Pheromone trail list	Nest resource tree (BST)
14	Space Colony	Settle most resource-rich tile first	Base layout satisfying distance rules	Settlement history list	Resource deposit tree (AVL)
15	Virus Defense	Patch most-connected vulnerable node first	Isolate cluster without breaking network	Infection spread list	Node vulnerability tree (BST)
16	Museum Heist	Grab highest-value item near guard first	Escape route avoiding all camera zones	Movement log list	Item value tree (BST)

Role Distribution Within the Team

Each team of three students must divide responsibilities as follows. All members are expected to understand the full project for the final demonstration:

Role	**Language**	**Responsibilities**
Engine Developer	C++	Implements linked list and BST/AVL tree. Builds `main.cpp` engine that processes `input.json` and produces `state.json`.
Algorithm Developer	Python	Implements greedy and backtracking functions in `greedy.py` and `backtracking.py`. Integrates with bridge layer.
UI Developer	Python + Pygame	Builds the Pygame grid, HUD, and animations in `main.py`. Implements `bridge.py` for file I/O integration.

Milestone 1 — Checkpoint (Week 8)

No executable code is required at this checkpoint. The team must submit a single PDF document containing:

1. **Algorithm Design:** Pseudocode for both the greedy and backtracking functions, specific to the team's game variant.
2. **Data Structure Design:** Diagram of the C++ linked list node structure and the BST or AVL tree node structure, with justification for using BST vs. AVL.
3. **File I/O Contract:** The exact JSON schema agreed upon for `input.json` and `state.json`, with field names, types, and example values.
4. **Pygame Wireframe:** A hand-drawn or digital sketch of the game grid layout and HUD elements.
5. **Work Split Statement:** A table declaring which team member owns each role, with a brief description of their planned contributions.

Final Deliverable (Week 16)

The team must submit:

- **Working Game:** A fully playable version of the game demonstrating all required components. The game must run from a single entry point (`game/ui/main.py`).
- **GitHub Repository:** A public or shared repository following the required folder structure, with a descriptive `README.md` explaining setup and execution instructions.
- **Technical Report (PDF):** A document covering:
  - Description of each algorithm implemented (greedy and backtracking) with pseudocode and complexity analysis.
  - Description of each data structure implemented in C++ with diagrams and operation complexity.
  - Explanation of the JSON bridge: how Python and C++ communicate.
  - Reflections on design decisions, challenges faced, and lessons learned.
- **Live Demo:** Each team presents their game in class during the final session (Week 16). All three members must be able to explain any part of the project.

Evaluation Criteria

- **C++ Engine:** Correctness and completeness of linked list and BST/AVL tree implementations. Proper reading and writing of JSON state files.
- **Python Algorithms:** Correct greedy and backtracking implementations that meaningfully drive game decisions. Proper integration with the bridge layer.
- **Pygame UI:** Functional grid-based interface. Clear visual feedback of algorithm decisions. Playable and responsive game loop.
- **Technical Report:** Clarity, correctness, and depth of documentation. Proper complexity analysis of implemented algorithms and structures.
- **Live Demo:** Team's ability to present and explain the full project. All members must demonstrate understanding of all layers.

Technical Requirements and Constraints

- The C++ engine must be compiled and executed as a standalone binary. No external C++ libraries beyond the standard library are allowed.
- The Python layer must use **Pygame** for all UI rendering. The `pygame` and standard library packages are the only allowed Python dependencies for the UI.
- The integration between C++ and Python must be done **exclusively through JSON file I/O**. No sockets, shared memory, or Python-C++ bindings are allowed.
- All source files must include a header comment with team number, variant name, and student names.
- The game grid must be at least 8×8 cells.
- The C++ linked list and tree must be implemented from scratch. Using `std::list`, `std::map`, `std::set`, or any equivalent STL container for the required structures is **not allowed**.

Self-Learning Resources

Students are expected to independently learn the following tools. Reference materials will be provided as a class example:

- **Pygame:** `https://www.pygame.org/docs/` — Official documentation. A working grid-based example will be provided in class using the Sort Race demo.
- **JSON in C++:** Use `nlohmann/json` header-only library (`https://github.com/nlohmann/json`) or manual string parsing. The class example will demonstrate both approaches.
- **JSON in Python:** Built-in `json` module — `https://docs.python.org/3/library/json.html`.
- **GitHub:** All teams must use version control from Week 1 of the project. Commit history will be reviewed as part of evaluation.

Academic Integrity

- All code must be written by the team. The use of AI-generated code that the team cannot explain is considered academic dishonesty.
- Code similarity between teams will be automatically checked. Teams with matching implementations will receive a grade of zero on the affected components.
- Each student is individually responsible for understanding and being able to explain *all* parts of the project, not only their assigned role.

This project is designed to be challenging, creative, and genuinely fun. The game is yours to build — the algorithms and data structures are the tools you have learned. Make the most of both halves of the course, and enjoy the process of bringing Computer Sciences I concepts to life in something you can actually play.